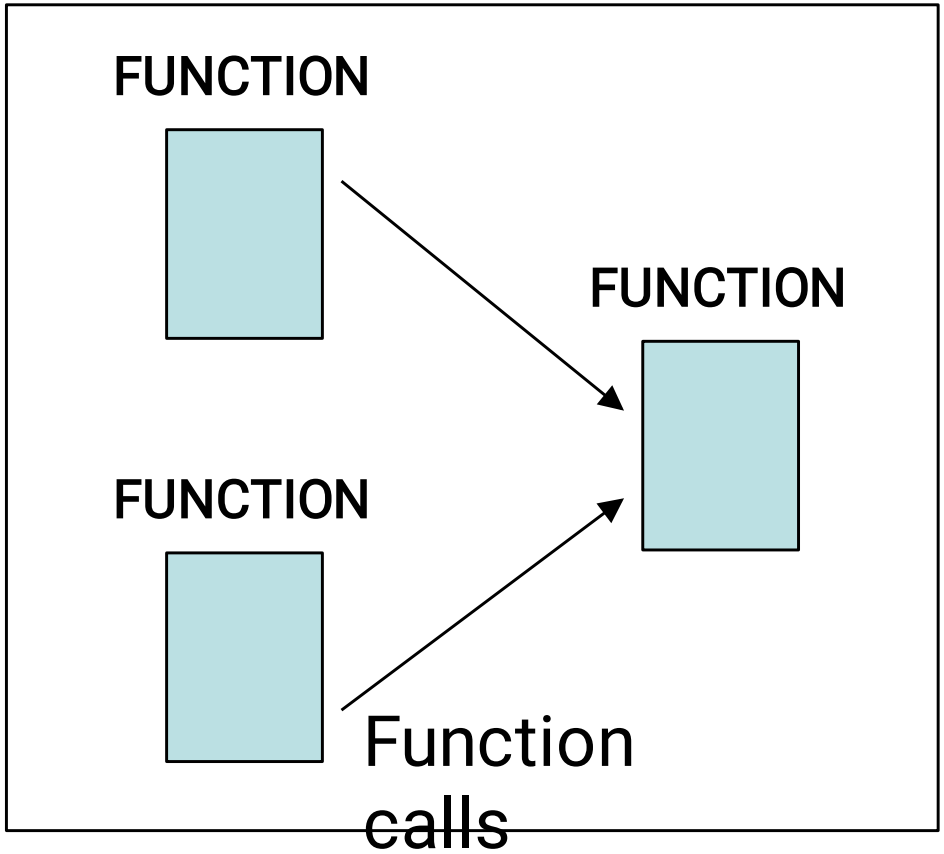
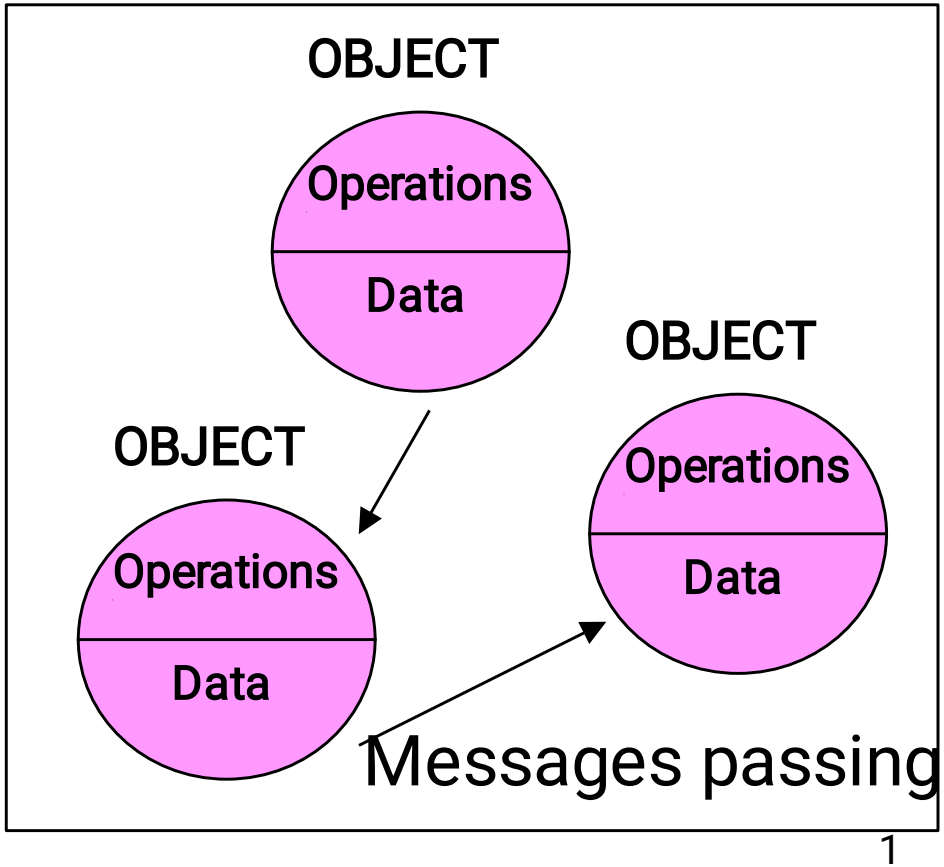
# Review:
# Two Programming Paradigms

**Structural (Procedural) PROGRAM**




**Object-Oriented PROGRAM**



# Review: Object-Oriented Programming Language Features

1. Data abstraction
2. Inheritance of properties
3. Dynamic binding of operations to objects

# Object-Oriented Programming--Introduction to Classes

- Class Definition
- Class Examples
- Objects

# Classes & Objects

- The class is the cornerstone of C++
  - It gives the C++ its identity from C
  - It makes possible encapsulation, data hiding and inheritance
- Class:
  - Consists of both data and methods
  - Defines properties and behavior of a set of entities
- Object:
  - An instance of a class
  - A variable identified by a unique name

# Classes & Objects

```
class Rectangle
{
  private:
    int width;
    int length;
  public:
    void set(int w, int l);
    int area();
}
```

```
Rectangle  r1;
Rectangle  r2;
Rectangle  r3;
    ⋮
```

# Define a Class Type

Header

```
class class_name
{
    permission_label:
        member;
    permission_label:
        member;
    ...
};
```

Body

```
class Rectangle
{
    private:
        int width;
        int length;
    public:
        void set(int w, int l);
        int area();
};
```

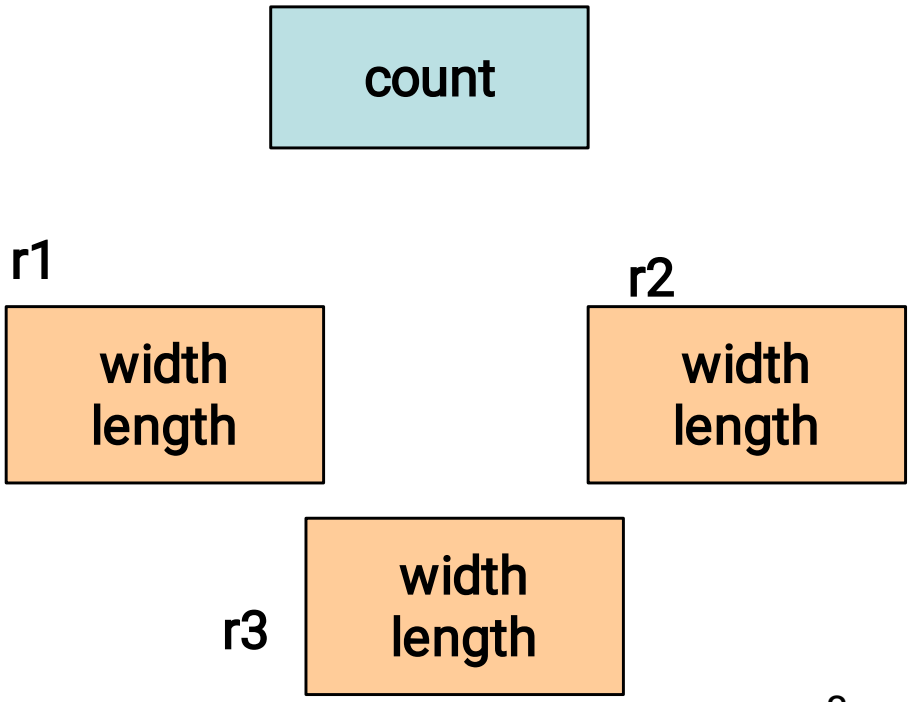
# Class Definition-Data Members

- Abstract the common attributes of a group of entities, their values determine the state of an object
- Can be of any type, built-in or user-defined
- *non-static* data member
  - Each class object has its own copy
  - Cannot be initialized explicitly in the class body
  - Can be initialized with member function, or class constructor
- *static* data member
  - Acts as a global object
  - One copy per class type, e.g. counter
  - Can be initialized explicitly in the class body

# Static Data Member

```
class Rectangle
{
  private:
    int width;
    int length;
  ➡ static int count;
  public:
    void set(int w, int l);
    int area();
  }
```

```
Rectangle  r1;
Rectangle  r2;
Rectangle  r3;
```



# Class Definition – Member Functions

- Used to
  - access the values of the data members (accessor)
  - perform operations on the data members (implementor)
- Are declared inside the class body, in the same way as declaring a function
- Their definition can be placed inside the class body, or outside the class body
- Can access both public and private members of the class
- Can be referred to using dot or arrow member access operator

# Define a Member Function

```
class Rectangle
{
   private:
      int width, length;
   public:
      void set (int w, int l);
      int area() {return width*length; }
}
```

inline

```
r1.set(5,8);

rp->set(8,10);
```

class name

member function name

```
void Rectangle :: set (int w, int l)
{
   width = w;
   length = l;
}
```

scope operator

# Class Definition – Member Functions

- **const** member function
  - declaration
    - *return_type func_name* (*para_list*) const;
  - definition
    - *return_type func_name* (*para_list*) const { … }
    - *return_type class_name* :: *func_name* (*para_list*) const { … }
  - Makes no modification about the data members (safe function)
  - It is illegal for a const member function to modify a class data member

# Const Member Function

```
class  Time
{
  private :
    int     hrs, mins, secs ;

  public :
    void    Write ( )  const ;
} ;
```

function declaration

function definition

```
void Time :: Write( ) const
{
    cout <<hrs << ":" << mins << ":" << secs << endl;
}
```

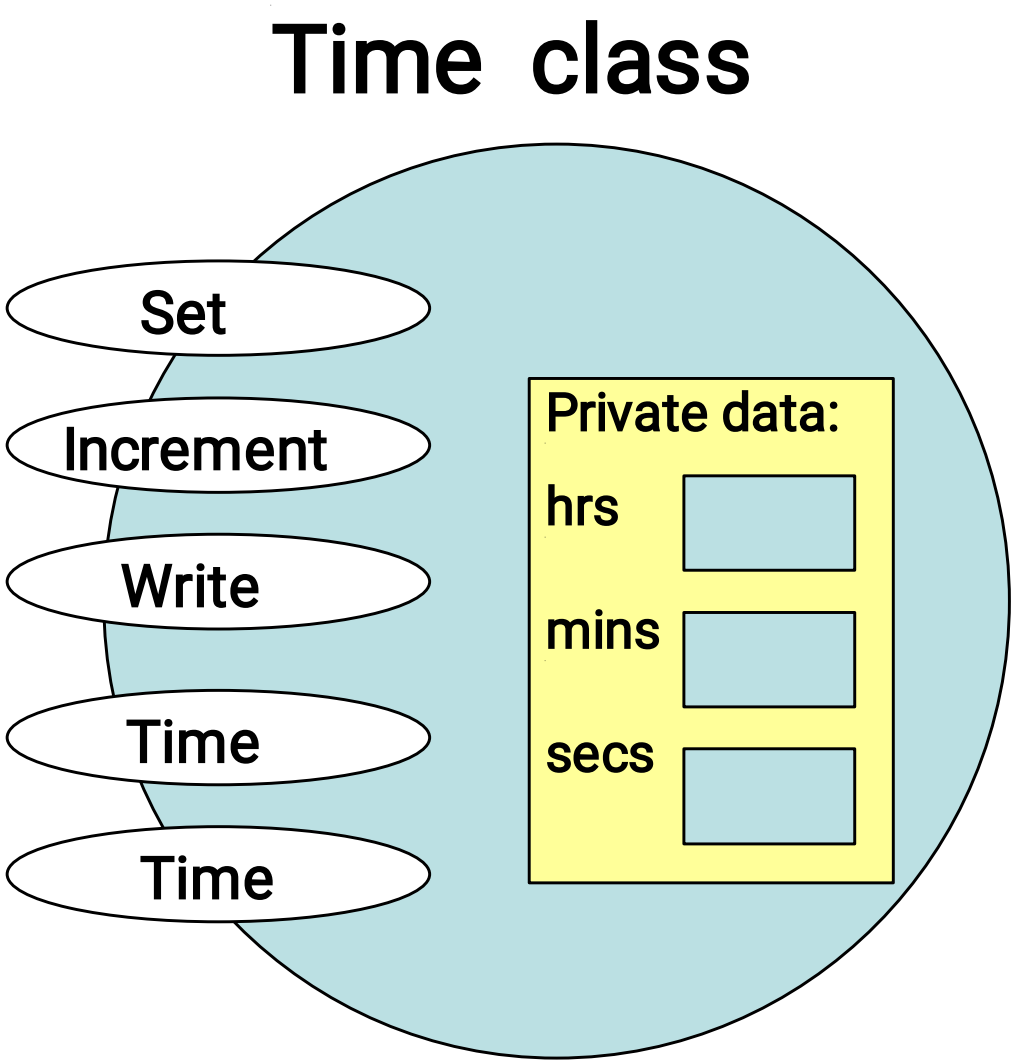
# Class Definition - Access Control

- Information hiding
  - To prevent the internal representation from direct access from outside the class
- Access Specifiers
  - public
    - may be accessible from anywhere within a program
  - private
    - may be accessed only by the member functions, and friends of this class
  - protected
    - acts as public for derived classes
    - behaves as private for the rest of the program

# class Time Specification

```
class  Time
{
  public :
    void    Set ( int  hours , int  minutes , int  seconds ) ;
    void    Increment ( ) ;
    void    Write ( )  const ;
    Time    ( int  initHrs, int  initMins,  int  initSecs ) ;  // constructor
    Time    ( ) ;                              // default constructor
  private :
    int          hrs ;
    int          mins ;
    int          secs ;
} ;
```

# Class Interface Diagram



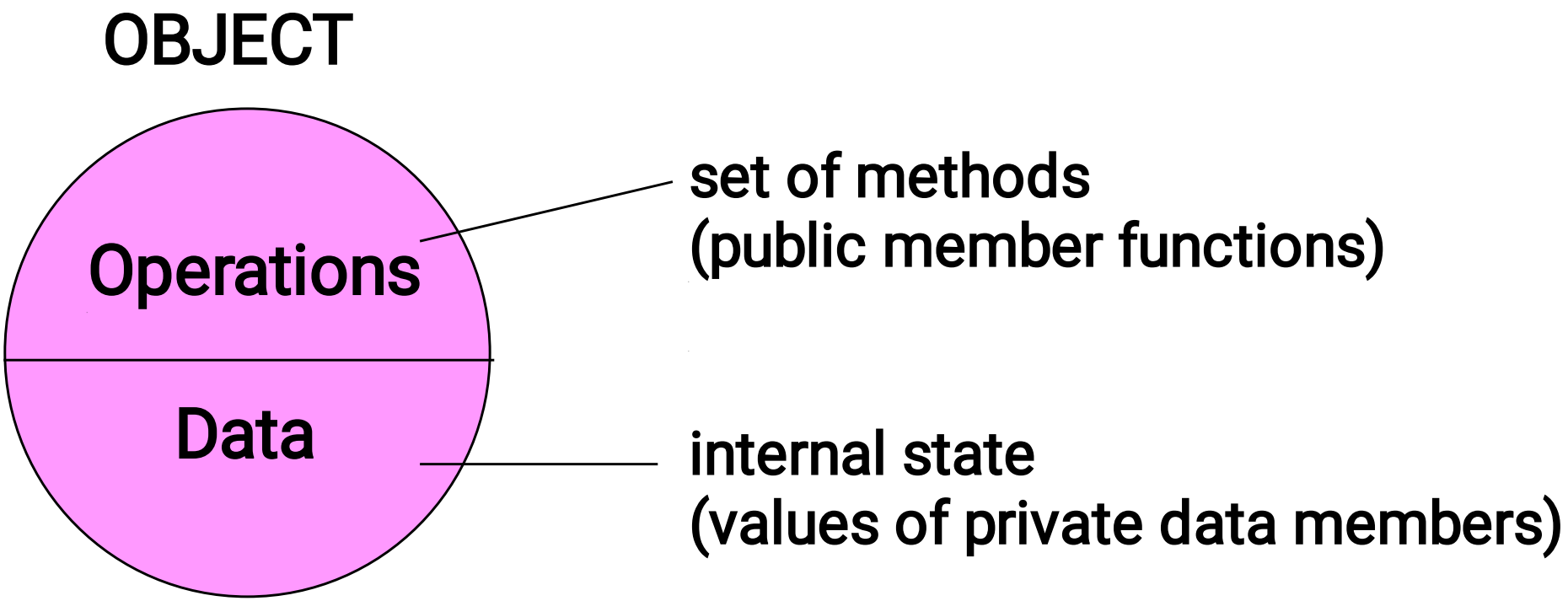
# Class Definition - Access Control

- The default access specifier is *private*
- The data members are usually private or protected
- A **private** member function is a helper, may only be accessed by another member function of the same class (exception *friend* function)
- The **public** member functions are part of the *class interface*
- Each access control section is optional, repeatable, and sections may occur in any order
- Difference between classes and structs in C++
  - the default access specifier is **private** in classes
  - the default access specifier is **public** in structs

# Objects

- Object:
  - a variable or an instance of a class
- Declaration of an Object
- Initiation of an Object

# What is an object?



# Declaration of an Object

```
class Rectangle
{
  private:
    int width;
    int length;
  public:
    void set(int w, int l);
    int area();
}
```

```
main()
{
   Rectangle r1;
   Rectangle r2;

    r1.set(5, 8);
   cout<<r1.area()<<endl;

   r2.set(8,10);
   cout<<r2.area()<<endl;
}
```

# Another Example

```
#include <iostream.h>

class circle
{
   private:
      double radius;

   public:
      void store(double);
      double area(void);
      void display(void);

};
```

```
// member function definitions

void circle::store(double r)
{
   radius = r;
}
double circle::area(void)
{
   return 3.14*radius*radius;
}
void circle::display(void)
{
   cout << “r = “ << radius << endl;
}
```

```
int main(void) {
    circle c;   // an object of circle class
    c.store(5.0);
    cout << "The area of circle c is " << c.area() << endl;
    c.display();
}
```

# Take Home Message

- Class can be considered as a user-defined data type, while an object is just a variable of certain class.

- There are three parts in the definition of a class: data members, member functions, and access control.